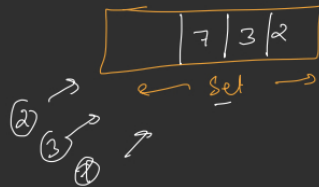


Set / Map

- ① Duplicates are not allowed.
- ② Constant time search operation.



set.add(2)
set.add(3)

set.contains(2)
→ $O(1)$

0	1	2	3	4
2	3	7		

$O(N)$

Website

↳ Aditya ✓
↳ Rohit ✓

Set

emailId = [x@gmail.com]

Maps

- (Key, value)
- Duplicate Key are not allowed
- Constant time search

(Roll No, Name)

(101, Aditya)
(102, Rohit)
(103, Aditya)

103	102	101
Aditya	Rohit	Aditya

Map

map.put(101, Aditya)
map.put(102, —)

map.containsKey(101);

→ $O(1)$ Y/N

map.get(102)

→ $O(1)$

Set / Map

↳ ① positional access.

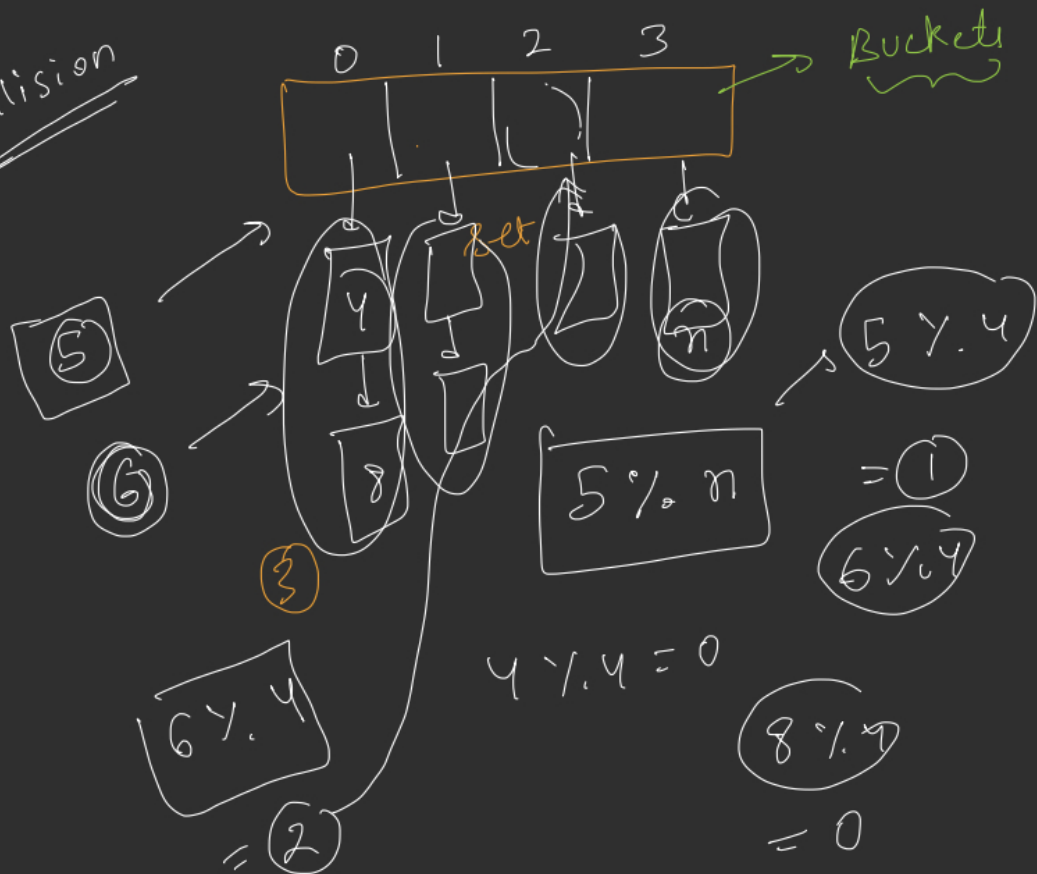
list =

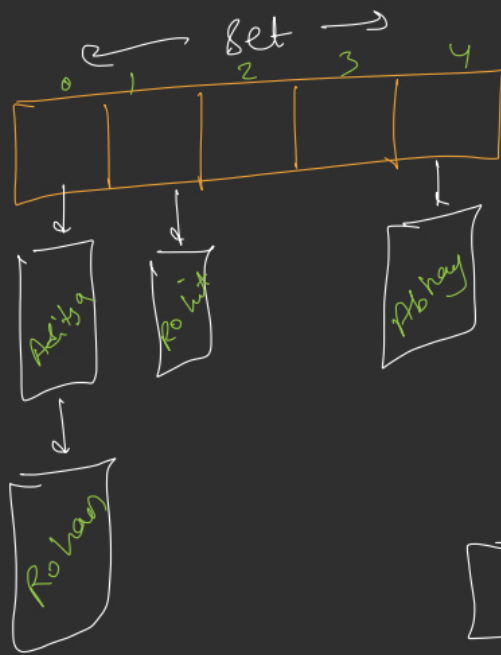
2	3	4	5
---	---	---	---

 0 1 2 3

⑤

Collision





$n=5$

Aditya, Rohit

hashCode()

Map

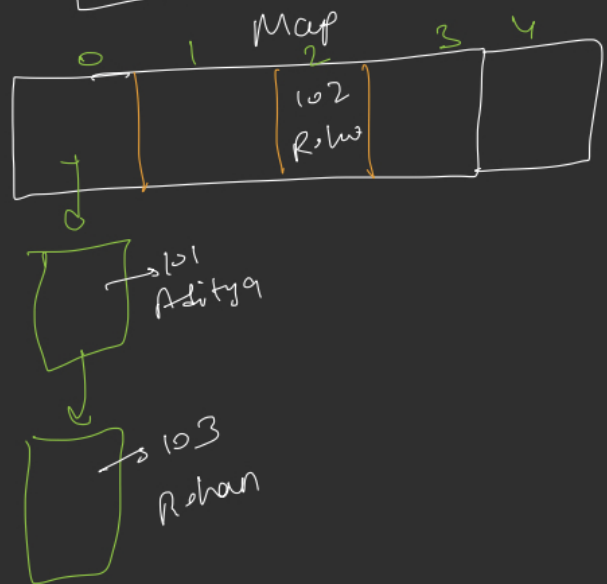
↳ (K, V)

↳ class

```
class Node <K, V> {
    K key;
    V value;
}
```

map.put(101, Aditya)

map.containsKey(101);
get(101);



Set / Map

Java → Set (X)

Set

- ↳ Duplicate (X)
- ↳ Constant time (✓)

Map ✓

- ↳ Duplicate X
- ↳ Constant X ✓

Set < Integer > set = new HashSet < > ();

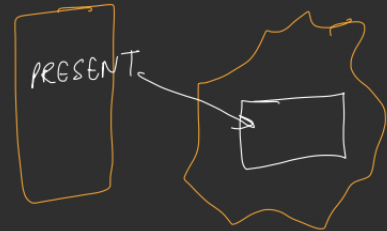
private HashMap < Integer, Object >
map = new HashMap ();

set.add(2);

map.put(2, PRESENT);

set.contains(2);

map.containsKey(2);



↳ Private static Final
PRESENT = new Object();

Java (Set/Map $\rightarrow$ Map)
 ↳ ① Key $\rightarrow$ hashCode (K, v)
 ↳ ② Bucket No. (Y)
 ↳ ③ Store in Bucket $\leftrightarrow$ ✓✓

Set/Map $\rightarrow$ Map

```

class Node<K, V> {
    K key;
    V value;
    int hash;
    Node<K, V> next;
}
  
```

Class HashMap {
 static class Node {
 }
 }

Set<String> s = new HashSet<>();

s.add("Aditya")

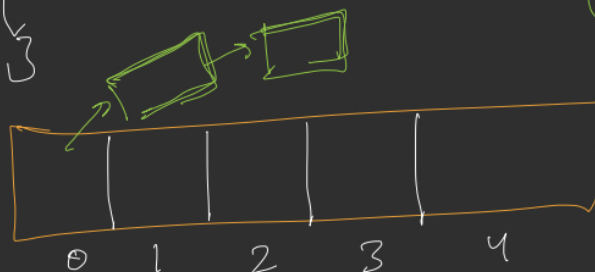
Set {
map
 PRESENT
 }

class Map
put()

boolean add(String Key) {
 (map.put(Key, PRESENT)
 == null);
 }

put (Key, value) {
 int (i) = Key.hashCode();
index = (i % n)
 }

n = 5



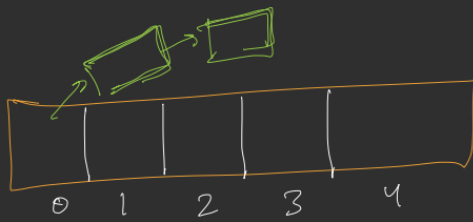
Case 1: Bucket Empty

bucket[0]
↳ (Aditya, PRESENT)

$\text{bucket}[\text{index}] = \text{new Node}(\text{Aditya}, \text{PRESENT}, i, \text{null});$

Case 2: Bucket Not Empty :

(Aditya, PRESENT, i, null)



(Aditya) n1

(Aditya) n2

Case 2.1 (Key already present)

↳

$n1 \neq n2$

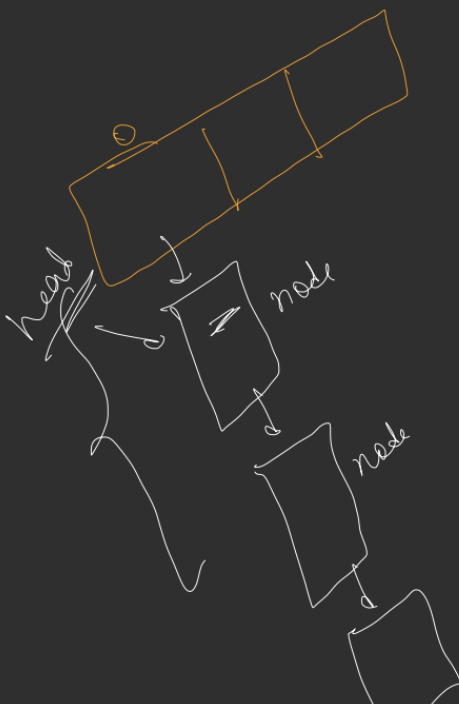
$(n1.v, n) = 0$

$(n2.v, n) = 0$

B[0]

(Aditya, PRESENT, ^P1, null)

$n1 == n2$



temp = head;

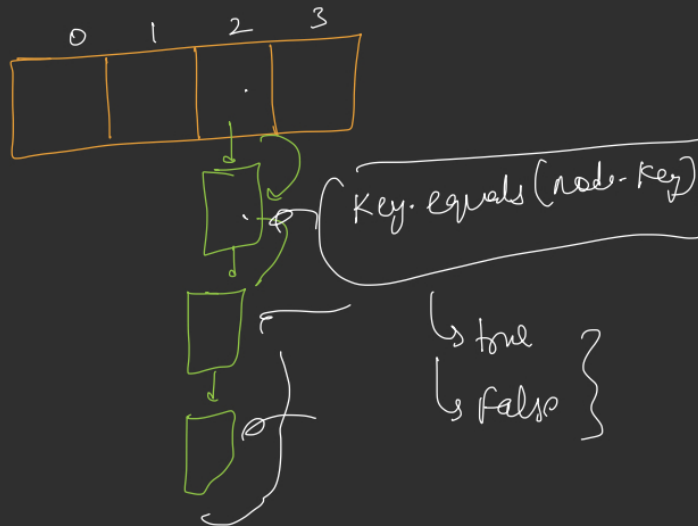
while (temp != null) {
 if (temp.hash == hash &&
 temp.Key.equals(Key) {
 // Duplicate Key
 }

else {

new node(K, v, i, null);

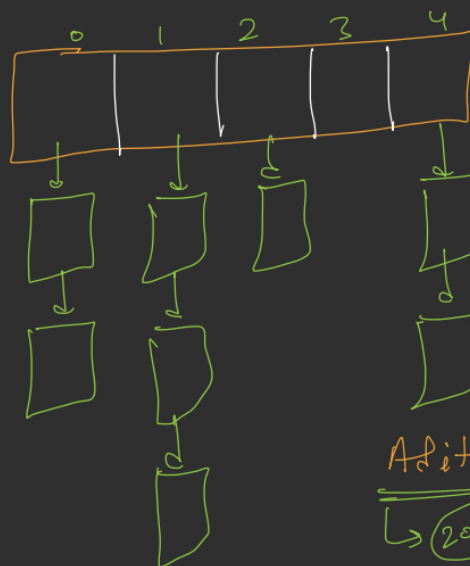
set.contains(Aditya);

↓
contains(Key) {
 map.containsKey(Key);
}



map.put(K, V);

↓
(101, Aditya)
↓
(hashCode() % n)
↓
equals()



Aditya
→ 2023

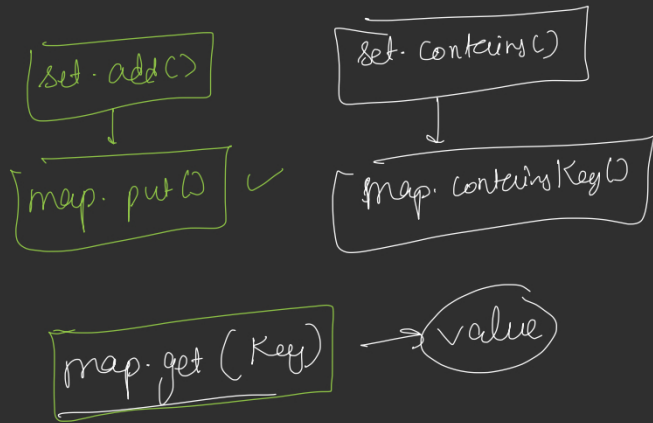
Rohit
→ 2023

equals()

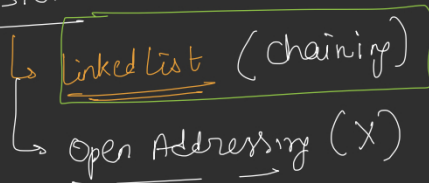
hashCode()

if (node.hash ==
hash
==)

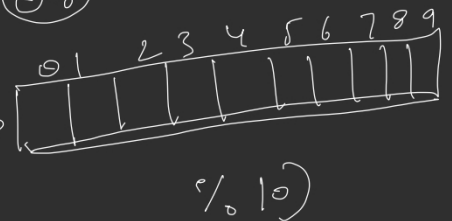
node.Key.equals(
Key)



Collision



$$(4/5) = 0.8$$

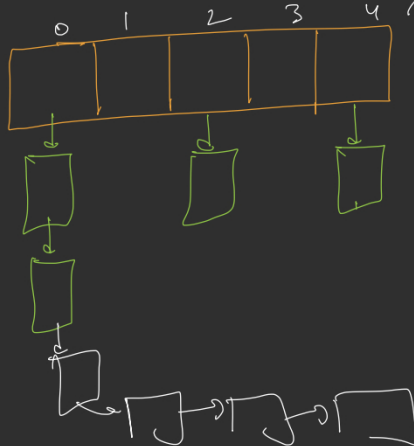


Load factor

elements = ? (6)
Capacity = ? (5)

$$\Rightarrow (\text{elements} / \text{Capacity})$$

$$0.75$$



(1) Create 2x as array.

Java $\rightarrow$ (16) capacity.

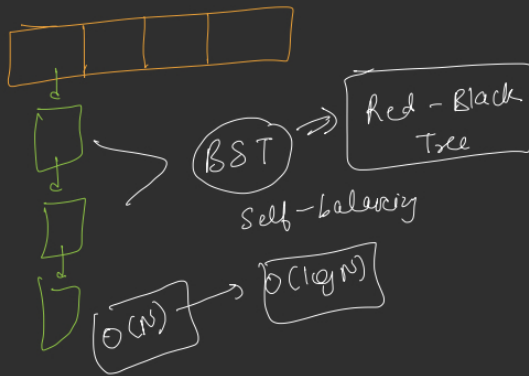
$$L-f = \frac{\text{elements}}{\text{Capacity}} < \underline{\underline{0.75}}$$

$\text{elements} < 12$ $\checkmark\checkmark$

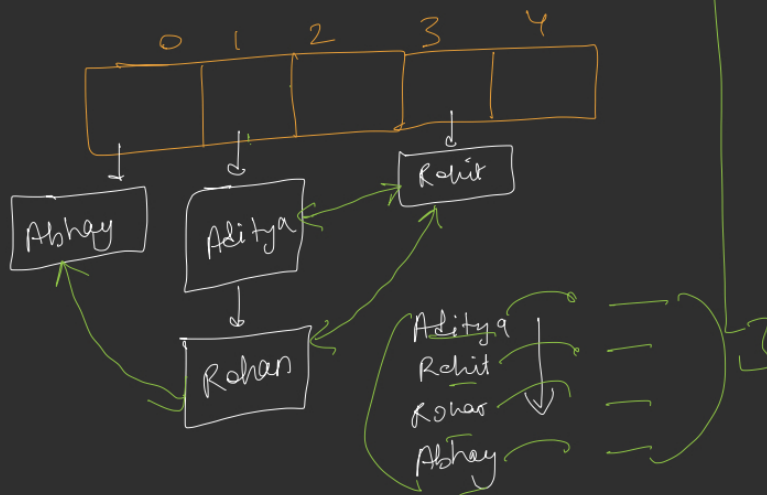
Tree Fication (Java 8 +)

ASY.

Prmsd ⑧



Linked Hash Set / Linked Hash Map



Class Node {

key,
value,
hash,
next;

before, after

TreeMap / TreeSet

(1, 10, 20, 30,
70, 80)

Self Balancing BST

(Red-Black Tree)

⇒ BST

→ Keys sorted

→ largest

→ smallest

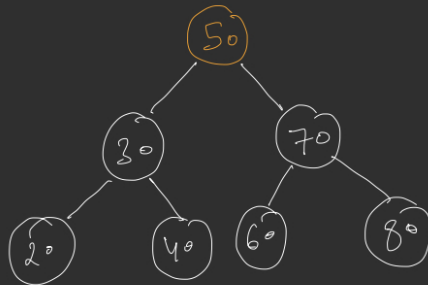
→ Range query



Aditya
Rohit
Rohan

BST

↳ smaller value goes to left.
↳ Higher value goes to right.



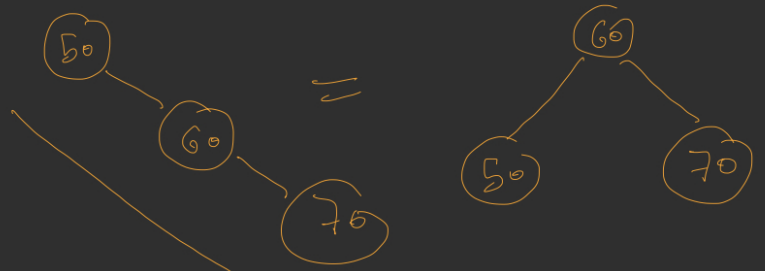
TreeSet

↓
TreeMap

(50, 70, 30, 20)
40, 60, 80

set.contains(70)

50, 60, 70, 80, 90



Class Node <K, V> {

K Key;

V value;

Node left;

Node right;

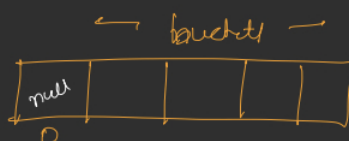
Node parent;

boolean color // TLF

Red-Black BST
AVL Tree

HashMap / HashSet

LinkedHashMap / LinkedHashSet



null → Key (1-time)

null → value (∞ time)

TreeMap / TreeSet → compare
= null X

